Algorithms

BASIC CONCEPTS

Overview of System Lifecycle

Insight - Tools and techniques that are necessary to design and implement large-scale computer programs.

A solid foundation in data abstraction, algorithm specification, and performance analysis and measurement provides the necessary methodologies

Recursive Algorithms

Good programmers regard large-scale computer programs as systems that contain many complex interacting parts.

- Face Book, Google Search engine, Amazon, Ticket Booking System

As systems, these programs undergo a development process called the system life cycle which consists of requirements, analysis, design, coding, and verification phases.

Although these phases are considered separately, they are highly interrelated and follow only a very crude sequential time frame.

UNIT 1

System Life Cycle
Algorithm - Definition
Criteria
Basic Steps in Algorithm Development
Algorithm Specification:
Pseudo-code Conventions
Recursive Algorithms.
Performance Analysis:
Space Complexity
Time Complexity
Asymptotic Notation.

SYSTEM LIFE CYCLE

Requirements

- All large programming projects begin with a set of specifications that define the purpose of the project.
- These requirements describe the information which has to be given to the programmers(input) and the results that the programmers must produce (output).

Analysis

- After the system' s requirements are carefully delineated, the analysis phase begins in earnest.
- In this phase, problems are break down into manageable pieces.
- Two approaches are used for breaking up the problem
  - Bottom-up
  - Top-down.
- Bottom-up methods are **unstructured strategies** that place an early emphasis on coding fine points.
  - Since the programmer does not have a master plan for the project, the resulting program frequently has many **loosely connected**, **error-ridden** segments.
  - Bottom-up analysis is akin to **constructing a building from a generic blueprint.**

- Top-up methods begin with the **purpose** that the **program** will serve & use this end-product to **divide** the **program** into **manageable-segments**

  This technique generates diagrams that are used to design the system.
- Frequently, **several alternate solutions** to the programming problem are **developed** and **compared** during this phase.

Design

- Designer approaches the system from perspectives of both

  The data objects that the program needs and

  The operations performed on them.
- The first perspective leads to the creation of abstract data types
- The second perspective requires

  The specification of algorithms and

  A consideration of algorithm design strategies.
- For example, suppose that we are designing a

  scheduling system for a university, Online shopping System-Amazon.
- Typical **data objects** might include students, courses, and professors.
- Typical **operations** might include inserting, removing, and searching within each object or between them.

SYSTEM LIFE CYCLE

Online shopping System,

- Typical **data objects** might include admin, product, customer, guest, cart and payment.
- Typical **operations** might include inserting, removing, and searching within each object or between them.

  Inserting, removing, updating, searching products & customers, viewing product, Add to cart, delete from cart, buying product, making payment
- Since the **abstract data types** and the **algorithm specifications** are language independent, implementation decisions may be postponed.
- **Coding details** may be **ignored** but the **information** required for each **data object** must be **specified**
- For ex, we might decide that the **student data object** should include name, social security number, major, and phone number.
- The **product data object** should include product_id, name, group, sub-group, qty, price,
- However, we would not yet pick a specific implementation for the list of students or products (arrays, linked lists, or trees).

Refinement and Coding

- In this phase,
  - The representations for data objects are **selected** and
  - Write the algorithms for each **operation** on them.
- The order is crucial because a data object' s **representation** can **determine** the **efficiency** of the algorithms related to it.

Verification

- Program Proving - developing correctness proofs for the program
- Testing - testing program with a variety of input-data
- Debugging - removing errors

Design Phase - Data Structures & Algorithms

Data Objects - Abstract Data Type - 1) DSA, 2) ADS

Operation - Algorithm Specification & Design Strategies - 3) DAA

Algorithm

Concept of algorithm is **fundamental** to computer science.

Algorithms are the **backbone** of **programming** and **data structures**.

If we understand algorithms, learning any programming language becomes **easy**."

Algorithms exits for many common problems, but **designing efficient algorithms** plays a **crucial role** in developing large scale computer systems.

ALGORITHM

Definition

An ***algorithm*** is a finite set of instructions that, if followed, accomplishes a particular task.

Ex : Manufacturing

- input: raw materials
- steps: process or procedure
- output: final Product

Similarly, in programming:

- input: data
- steps: algorithm
- output: result”

as

Criteria

- Input : 0 or 1 quantities are externally supplied.
- Output : atleast 1 quantity is produced.
- Definiteness :each instruction is **clear and unambiguous**
- Finiteness : If we trace out the instructions of an algorithm, then **for all cases,** the algorithm terminates after a finite number of steps
- Effectiveness: instruction should be basic enough and feasible to carry out

Algorithm Vs Program

Program- does not statisfy 4^{th} criteria

Ex : An OS that continues in a wait loop until more jobs are entered.

- Such a program does not terminate unless the system crashes.

Since all our programs will always terminate, we will use algorithm and program interchangeably.

Basic Steps in Algorithm Development

How an algorithm is actually developed?

Before writing any program, first design an algorithm.

These basic steps(are the following) help to convert a problem into a working solution."

Step 1: Problem definition

Step 2: Analyze the problem

Step 3: Design the algorithm

Step 4: Test the algorithm manually

Step 5: Implement using programming language

Step 6: Evaluate and refine

Step 1: Problem definition

Clearly understand the problem
Identify objectives of the solution
Specify required inputs and expected outputs
Note constraints and assumptions
Ex Problem: find the largest of three numbers
Inputs → three numbers
Output → largest number
A well-defined problem leads to a correct algorithm.”

Step 2: Analyze the problem

“Next, we analyze the problem in depth.
Break the problem into smaller parts
Identify data to be used
Determine operations to be performed
Check feasibility of solution in terms of time and resources

Step 3: Design the algorithm

"In this step, we actually design the algorithm.
Develop step-by-step procedure
Use simple and unambiguous statements
Choose representation: pseudocode or flowchart
Ensure each step is logically ordered

STEP 4: TEST THE ALGORITHM MANUALLY

"After designing, we test the algorithm before coding. This is called dry run or desk checking.

Test with sample input values

Verify correctness of output

Check boundary conditions(like maximum & minimum values)and special cases

Modify the algorithm if errors are found

Example values:
3, 7, 5 → output should be 7

Step 5: Implement using programming language

"In this step, the algorithm is converted into a real program. Choose a programming language like C, Java, or Python.

Convert algorithm into a programming language
Select/Use appropriate data structures
Write, compile and execute the program

Step 6: Evaluate and refine

“Finally, we evaluate how good the algorithm is.

Analyze time and space efficiency
Remove unnecessary steps
Simplify logic where possible
Optimize performance

If the algorithm is slow or complex, we refine and optimize it.

Improvement is an ongoing process—algorithms are often revised many times.”

SUMMARY

First define the problem, then analyze it, design the algorithm, test it, implement it, and finally refine it.

Define → Analyze → Design → Test → Implement → Refine

Algorithm development is **iterative-**we may go back and modify earlier steps

Strong & Good algorithm design leads to simple, efficient, and correct programs."

Algorithm to find largest of two numbers

Read A and B

If A > B
print A is largest
else
print B is largest

Stop

Description of algorithm can be done in many ways

- Natural Language(mathematical equations) : English – but make sure the resulting instructions are definite
- Graphical Representations : Flowcharts – but will work well only if the algorithm is small and simple
- Combination of C & English Language constructs.

Two examples – which will illustrate the process of translating a problem into an algorithm.

EXAMPLE 1 – SELECTION SORT

Suppose we must devise a program that sorts a set of n>=1 integers. A simple solution is given by the following

From the given integers that are currently unsorted, find the smallest and place it next in the sorted list.

Although this statement adequately describes the sorting problem, it is not an algorithm since it leaves several unanswered questions.

For example, it does not tell us where and how the integers are initially stored, or where we should place the result.

We assume that the integers are stored in an array, *list,* such that the ith integer is stored in the ith position, *list* [i], 0 < *i* < *n.*

SELECTION SORT

Program 1.1 is our first attempt at deriving a solution. Notice that it is written partially in C and partially in English.

```
for (i = 0; i < n; i++) {
  Examine list[i] to list[n-1] and suppose that the
  smallest integer is  at list[min];

  Interchange list[i] and list[min];
}
```

Program 1.1: Selection sort algorithm

To turn Program 1.1 into a real C program, two clearly defined subtasks remain:

- finding the smallest integer and
- interchanging it with *list[i].*

We can solve the latter problem using either a function (Program 1.2) or a macro.

The function's code is easier to read than that of the macro but the macro works with any data type.

Using the function, suppose *a* and *b* are declared as **ints.**

To swap their values one would say:

```
swap (&a, &;b) ;
```

```
void swap(int *x, int *y)
/* both parameters are pointers to ints */
{
   int temp = *x;    /* declares temp as an int and assigns
                        to it the contents of what x points to */
   *x = *y; /* stores what y points to into the location
                                          where x points */
   *y = temp; /*places the contents of temp in location
                                        pointed to by y */
}
```

Program 1.2: Swap function

We can solve the first subtask by assuming that the minimum is *list[i]*, checking *list[i]* with list[i+1], list[i+2], • • • , *list[n-1]*.

Whenever we find a smaller number we make it the new minimum.

When we reach *list[n-1]* we are finished.

Putting all these observations together gives us *sort* (Program 1.3).

Program 1.3 contains a complete program which you may run on your computer.

All programs in this book follow the rules of ANSI C.

```c
#include <stdio.h>
#include <math.h>
#define MAX_SIZE 101
#define SWAP(x,y,t) ((t) = (x), (x)= (y), (y) = (t))
void sort(int [],int); /*selection sort */
void main(void)
{
   int i,n;
   int list[MAX_SIZE];
   printf("Enter the number of numbers to generate: ");
   scanf("%d",&n);
   if( n < 1 || n > MAX_SIZE) {
     fprintf(stderr, "Improper value of n\n");
     exit(1);
   }
   for (i = 0; i < n; i++) {/*randomly generate numbers*/
      list[i] = rand() % 1000;
      printf("%d  ",list[i]);
   }
   sort(list,n);
   printf("\n Sorted array:\n ");
   for (i = 0; i < n; i++) /* print out sorted numbers */
      printf("%d  ",list[i]);
   printf("\n");
}
void sort(int list[],int n)
{
   int i, j, min, temp;
   for (i = 0; i < n-1; i++)   {
      min = i;
      for (j = i+1; j < n; j++)
         if (list[j] < list[min])
            min = j;
      SWAP(list[i],list[min],temp);
   }
}
```

Program 1.3: Selection sort

EXAMPLE 2 : BINARY SEARCH

Assume that we have *n* > 1 **distinct integers** that are already **sorted** and stored in the array *list.*

That is , list[0]< list[1] < • • • < *list[n-1].*

Find out if an integer ***searchnum*** is present in this list or not.

If it is, return an index, i, such that ***list[i] = searchnum.***

If *searchnum* is **not present**, **return -1.**

Since the list is **sorted** we may use the following method to search for the value.

Let ***left*** and ***right***, respectively, denote the left and right **ends** of the **list** to be **searched**.

Initially, ***left* = 0** and ***right* = n-l**.

Let ***middle = (left+right)/2*** be the middle position in the list.

If we **compare** ***list [middle*** **]** with ***searchnum***, we obtain one of three results:

(1) **searchnum < list[middle]**. In this case, if *searchnum* is present, it must be in the positions between 0 and *middle* – 1. Therefore, we set *right* to *middle* – 1.

(2) **searchnum = list[middle].** In this case, we return *middle*.

(3) **searchnum > list[middle].** In this case, if *searchnum* is present, it must be in the positions between *middle* + 1 and *n* – 1. So, we set *left* to *middle* + 1.

If ***searchnum*** has **not been found** and there are **still integers** to check, we **recalculate *middle*** and **continue** the **search**.

Program 1.4 implements this searching strategy.

The algorithm contains two subtasks:

- (1) determining if there are any integers left to check,

(2) comparing *searchnum* to *list[middle].*

```
while (there are more integers to check ) {
   middle = (left + right) / 2;
   if (searchnum < list[middle])
      right = middle - 1;
   else if (searchnum == list[middle])
            return middle;
         else left = middle + 1;
}
```

Program 1.4: Searching a sorted list

```
int compare(int x, int y)
{
/* compare x and y, return -1 for less than, 0 for equal,
1 for greater */
   if (x < y) return -1;
   else if (x == y) return 0;
        else return 1;
}
```

Program 1.5: Comparison of two integers

```
int binsearch(int list[], int searchnum, int left,
                                         int right)
{
/* search list[0] <= list[1] <=  · · ·  <= list[n-1] for
searchnum. Return its position if found. Otherwise
return -1 */
   int  middle;
   while (left <= right)  {
      middle = (left + right)/2;
      switch (COMPARE(list[middle], searchnum)) {
         case -1: left = middle + 1;
                  break;
         case 0 : return middle;
         case 1 : right = middle - 1;
      }
   }
   return -1;
}
```

Program 1.6: Searching an ordered list

- Algorithm 1.3: Binary search.

assumption :sorted n(≥1) distinct integers stored in the array *list*
return *i* if *list*[*i*] = *searchnum*;
-1 if no such index exists
denote *left* and *right* as left and right ends of the list to be searched (*left*=0 & *right*=n-1)
let *middle*=(*left*+*right*)/2 middle position in the list
compare list[middle] with searchnum and adjust left or right
compare list[middle] with searchnum
1) searchnum < list[middle]
set right to middle-1
2) searchnum = list[middle]
return middle
3) searchnum > list[middle]
set left to middle+1
if searchnum has not been found and there are more integers to check recalculate middle and continue search

(10, 20, 30, 40 50); n=5; left =0; right=n-1=5-1=4

Searchnum=40

Middle=(left+right)/2 = (0+4)/2=2

Compare list[2] with searchnum; 30 ? 40

If 30<40 set left =middle+1=2+1=3

Left=3 right=4

Middle=(left+right)/2 = (3+4)/2=7/2=3.5=3

Compare list[3] with searchnum; 40 ? 40

If 40<40 X

If 40=40 ✓

Return middle; 3

Binary Search *is a searching algorithm used in a sorted array by* ***repeatedly dividing the search interval in half****. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to* ***O(Log n)****.*

The basic steps to perform Binary Search are:

- **Begin** with the **mid element** of the **whole array** as **search key.**
- If the **value** of the **search key** is **equal** to the **item** then **return** index of the search key.
- Or if the **value** of the **search key** is **less than** the **item** in the middle of the interval, narrow the interval to the lower half.
- Otherwise, narrow it to the upper half.
- Repeatedly check from the second point **until** the **value** is **found** or the **interval** is **empty**.

Binary Search

Algorithm Specification – Pseudocode Conventions

Data Structures

Teaching Presentation

Topic Overview

WHAT IS ALGORITHM SPECIFICATION?

Before writing code, must be clear about what the **algorithm does**

Algorithms must be described in a **precise** but **language-independent** way

This description is called algorithm specification

AS means writing a **clear** and **unambiguous** description of an algorithm so that anyone can understand the **logic** without worrying about a specific programming language.

Clear and precise description of algorithm

Avoids ambiguity

Acts as blueprint before programming

WHAT IS PSEUDOCODE?

" Pseudocode is not an actual programming language.

It is not English paragraph writing

Structured, English-like representation of algorithm

Language independent

Def : It is a high-level, informal description of an algorithm that uses a mixture of natural language and programming-language-like constructs.

Why Use Pseudocode?

Independent of programming language

Easy to write than code

Easy to understand and discuss

Useful before real coding

Easy to convert into any programming language

focuses on logic, not syntax

avoids compiler rules, punctuation, data types etc.

useful for planning & Algorithm Analysis:

- flowcharts
- programs
- documentation

EXAMPLE

Task: Add two numbers

START

Read A, B

Compute SUM = A + B

Print SUM

STOP

PSEUDOCODE CONVENTIONS

"Even though pseudocode is informal, cannot write it randomly.

Follow certain **conventions or standards** standard writing style

Improves readability

Avoids confusion

CONVENTION 1 – UPPERCASE KEYWORDS

Write important control words like

- IF, THEN, ELSE, FOR, WHILE, REPEAT, UNTIL, BEGIN, END in **uppercase**

This visually distinguishes them from **variables** & **normal text**

Example:

```
IF A > B THEN
    PRINT A
ELSE
    PRINT B
END IF
```

Convention 2 – Indentation

Indent statements inside loops and conditions

Shows block structure clearly

It helps the reader see which statements belong inside the condition or loop.

If we do not indent, the pseudocode becomes confusing, especially in nested conditions.”

Example:

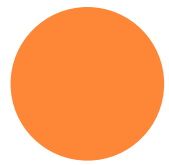
```
IF N > 0 THEN
    SUM = SUM + N
END IF
```

Convention 3 – Meaningful Variable Names

Avoid meaningless variable names like x1, y2 type, name

Instead use names like TOTAL, COUNT, MARKS, AVERAGE, STU_NAME etc.

The name should indicate the purpose of the variable so that another person can easily understand the algorithm.”



Convention 4 – Control Structures

"Every pseudocode uses only three basic control structures:

Sequence – step-by-step execution

Selection – decision making using IF THEN ELSE

Iteration – repetition using loops like FOR, WHILE

All algorithms, even very complex ones, are built from these three."

Sequence example:

```
READ A
READ B
SUM = A + B
PRINT SUM
```

Selection example:

```
IF MARK ⩾ 50 THEN
    PRINT "PASS"
ELSE
    PRINT "FAIL"
END IF
```

Iteration example:

```
FOR I = 1 TO N DO
    READ A[I]
END FOR
```

Convention 5 – Avoid Programming Syntax

"When writing pseudocode – avoid the exact syntax of C, C++ or Java:

No semicolons or curly braces

No datatype declarations

No header files

write in simple structured English with keywords.

Example:

❌ C code style → for(i=0;i<n;i++)

✔ Pseudocode → FOR i ← 1 TO n DO

CONVENTION 6 – BEGIN AND END

Often use BEGIN and END to mark the start and finish of an pseudocode

Improves clarity of algorithm boundaries

Inside BEGIN and END, we write all the pseudocode steps.”

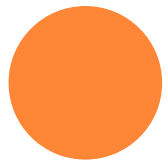
Example:

```
BEGIN
    READ N
    SUM = 0
    FOR I = 1 TO N DO
        SUM = SUM + I
    END FOR
    PRINT SUM
END
```

Example Pseudocode

Simple ex: finding the largest of three numbers.

Read three numbers
Compare values
Print largest number
Uses conventions discussed



Good pseudocode:

```
START
READ A, B, C
IF A ≥ B AND A ≥ C THEN
    PRINT A
ELSE IF B ≥ A AND B ≥ C THEN
    PRINT B
ELSE
    PRINT C
END IF
STOP
```

Common Mistakes

Mixing pseudocode with real code

Missing indentation

Not closing IF or loops

using vague English sentences instead of structured steps

not writing START and STOP/BEGIN and END

Avoiding these mistakes, pseudocode will be clear and professional.”

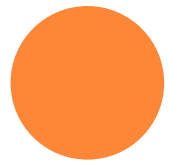
Practice Questions

Try writing pseudocode for a few simple problems(avoid syntax, focus on logic)

Check whether a number is EVEN or ODD

Find sum of first N natural numbers

Find factorial of N



RECURSIVE ALGORITHMS

A function calls itself either directly or indirectly during execution.

A function may call other functions that invoke the calling function again *(indirect recursion)*

Recursive-algorithms when compared to **iterative-algorithms** are normally compact and easy to understand.

These recursive mechanisms are not only extremely powerful, but frequently allows to express an **complex process** in very clear terms.

It is for these reasons that the recursion are introduced.

Frequently we regard recursion as a mystical technique that is useful for only a few special problems such as

- computing factorials or Ackermann's function.

This is unfortunate because any function that written using **assignment**, **if-else,** and **while** statements can be written recursively.

Often this **recursive function** is easier to understand than its **iterative** counterpart.

- Various types of recursion:

1) Direct recursion: where a recursive-function invokes itself.

For Ex,

```
int fact(int n)                 //to find factorial of a number
{
        if(n==0)
                return 1;
        return n*fact(n-1);
}
```

2) Indirect recursion: A function which contains a call to another function which in turn calls another function and so on and eventually calls the first function.

For Ex,

```
void f1()          void f2()          void f3()
{                  {                  {
  ....               ....               ....
  f2();              f3();              f1();
}                  }                  }
```

Recursion-Properties

To transform the function into a recursive one,

- (1) establish **boundary conditions** that **terminate** the **recursive calls**, and
- (2) **implement** the **recursive calls** so that each call brings us **one step closer** to a **solution**.

A recursive function can **go infinite** like a **loop**.

To **avoid infinite running** of recursive function, there are **two properties** that a recursive function must have –

- **Base criteria** – There must be **at least one base criteria or condition**, such that, when this **condition** is **met** the **function stops calling itself recursively.**
- **Progressive approach** – The recursive calls should progress in such a way that each time a recursive call is made it comes **closer** to the **base criteria**.

Two examples :1) Binary Search 2) Permutations

BINARY SEARCH

In **Binary search**, there are two ways to terminate:

- one signaling a success *(list[middle] = searchnum),*
- the other signaling a failure (the left and right indices cross).

No need to change the code when the function terminates successfully.

However, the **while** statement that is used to trigger the **unsuccessful search** needs to be replaced with an equivalent **if** statement whose **then** clause invokes the **function recursively**.

Creating recursive calls that move us closer to a solution is also simple since it requires only passing the new *left* or *right* index as a parameter in the next recursive call.

Program 1.5 implements the recursive binary search.

Notice that although the code has changed, the recursive function call is identical to that of the iterative function.

```
int binsearch(int list[], int searchnum, int left, int right)
{       // search list[0]<= list[1]<=...<=list[n-1] for searchnum
        int middle;
        if (left<= right)
        {
                middle= (left+ right)/2;
                switch(compare(list[middle], searchnum))
                {
                  case -1:return binsearch(list, searchnum, middle+1, right);
                  case 0: return middle;
                  case 1: return binsearch(list, searchnum, left, middle- 1);
                }
        }
        return -1;
}

int compare(int x, int y)
{
        if (x< y) return -1;
        else if (x== y) return 0;
             else return 1;
}
```

PERMUTATIONS

Permutations *:* Given a set of n > 1 elements, print out all possible permutations of this set.

For example, if the set is *[a, b.* c), then the set of permutations is {(a, *b, c), (a, c, b}, (b, a,* c), *{b, c, a}, {c, a, b}* {c, *b,* a}).

It is easy to see that, given *n* elements, there are n! permutations.

We can obtain a simple algorithm for generating the permutations if we look at the set *[a, b,* c, d).

We can construct the set of permutations by printing:

- *a* followed by all permutations of *(b,* c, *d)*
- *b* followed by all permutations of *(a,* c, *d}*
- *c* followed by all permutations of *{a, b, d}*
- *d* followed by all permutations of (a, *b, c}*

The clue to the recursive solution is the phrase "followed by all permutations."

It implies that we can solve the problem for a set with *n* elements if we have an algorithm that works on *n* - 1 elements.

These considerations lead to the development of Program 1.8.

We assume that *list* is a character array.

Notice that it recursively generates permutations until *i = n.*

The initial function call is *perm(list. 0, n-1);*

```
void perm(char *list,int i,int n)
{
        int j,temp;
        if(i==n)
        {
                for(j=0;j<=n;j++)
                printf("%c", list[j]);
                printf(" ");
        }
        else
        {
                for(j=i;j<=n;j++)
                {
                    SWAP(list[i],list[j],temp);
                    perm(list,i+1,n);
                    SWAP(list[i],list[j],temp);
                }
        }
}
```

Program 1.6: Recursive permutations generator

Towers of Hanoi

Towers of Hanoi There are three towers and 64 disks of different diameters placed on the first tower.

The disks are in order of decreasing diameter as one scans up the tower.

Monks were reputedly supposed to move the disk from tower 1 to tower 3 obeying the rules:

- Only one disk can be moved at any time.
- No disk can be placed on top of a disk with a smaller diameter.

Write a recursive function that prints out the sequence of moves needed to accomplish this task.

```
void Hanoi(int n, char x, char y, char z)
{
        if (n > 1)
         {
            Hanoi(n-1,x,z,y);
            printf("Move disk %d from %c to %c.\n",n,x,z);
            Hanoi(n-1,y,x,z);
        }
        else
        {
            printf("Move disk %d from %c to %c.\n",n,x,z);
        }
}
```

Program 1.7: Recursive Implementation of tower of Hanoi

Performance Analysis–Space and Time Complexity

PERFORMANCE ANALYSIS

Criteria for judging a program

- Does the program meet the original specification of the task?
- Does it work correctly?
- Does the program contain documentation that shows how to use it and how it works?
- Does the program effectively use functions to create logical units?
- Is the program’ s code readable?

Although the above criteria are vitally important, particularly in the development of large systems, it is difficult to explain how to achieve them.

More Concrete Criteria

- Does the program efficiently use primary and secondary storage?
- Is the program’ s running time acceptable for the task?

MEASUREMENTS

These criteria focus on performance evaluation, which is loosely divided into

- Performance Analysis (machine independent)
  - space complexity: storage requirement
  - time complexity: computing time
- Performance Measurement (machine dependent)

PERFORMANCE ANALYSIS

The process of estimating time & space consumed by program is called *performance analysis*.

Efficiency of a program depends on 2 factors:

- Space efficiency (primary & secondary memory) &
- Time efficiency (execution time of program)

SPACE COMPLEXITY

Space complexity of a program is the amount of memory required to run the program completely.

Total space requirement of any program is given by

- S(P)= fixed space requirement + variable space requirement
- S(P)= c + Sp(I)

Fixed Space Requirements

This component refers to space requirements that do not depend on the number and size of the program's inputs and outputs.

Fixed requirements include program space (space for storing machine language program) data space, space for simple variables and fixed – size component variables(also called aggregate), and space for constants

Variable Space Requirements

This component consists of space needed by structured(component) variables whose size depends on particular instance of problem being solved.

This also includes space needed by reference variables and additional space required when a function uses recursion.

SPACE COMPLEXITY

$$S(P)=C+S_P(I)$$

Fixed Space Requirements (C)

Independent of the characteristics of the inputs and outputs

- instruction space
- space for simple variables, fixed-size structured variable, constants

Variable Space Requirements ($S_P(I)$)

depend on the instance characteristic I

- number, size, values of inputs and outputs associated with I
- recursive stack space, formal parameters, local variables, return address
- S(P)=c+Sp(instance characteristics), where c is a constant

```
1    Algorithm abc(a, b, c)
2    {
3        return  a + b + b * c + (a + b − c)/(a + b) + 4.0;
4    }
```

Algorithm 1.5 Computes $a + b + b * c + (a + b - c)/(a + b) + 4.0$

Example 1.4 For Algorithm 1.5, the problem instance is characterized by the specific values of a, b, and c. Making the assumption that one word is adequate to store the values of each of a, b, c, and the result, we see that the space needed by abc is independent of the instance characteristics. Consequently, $S_P(\text{instance characteristics}) = 0$. □

```
1    Algorithm Sum(a, n)
2    {
3        s := 0.0;
4        for i := 1 to n do
5            s := s + a[i];
6        return s;
7    }
```

Algorithm 1.6 Iterative function for sum

Example 1.5 The problem instances for Algorithm 1.6 are characterized by n, the number of elements to be summed. The space needed by n is one word, since it is of type *integer*. The space needed by a is the space needed by variables of type array of floating point numbers. This is at least n words, since a must be large enough to hold the n elements to be summed. So, we obtain $S_{\mathsf{Sum}}(n) \geq (n+3)$ (n for $a[\,]$, one each for n, i, and s). □

```
1    Algorithm RSum(a, n)
2    {
3        if (n ≤ 0) then return 0.0;
4        else return RSum(a, n − 1) + a[n];
5    }
```

Algorithm 1.7 Recursive function for sum

Example 1.6 Let us consider the algorithm RSum (Algorithm 1.7). As in the case of Sum, the instances are characterized by n. The recursion stack space includes space for the formal parameters, the local variables, and the return address. Assume that the return address requires only one word of memory. Each call to RSum requires at least three words (including space for the values of n, the return address, and a pointer to $a[\,]$). Since the depth of recursion is $n+1$, the recursion stack space needed is $\geq 3(n+1)$. □

TIME COMPLEXITY

The time T(P) taken by a program P is the sum of the compile time and the run or execution time .

T(P)=compile time + run(or execution)time

The **compile time** is similar to the **fixed space component** since it does **not depend** on the **instance characteristics.**

Compiled Program will be run several times **without recompilation**

Consequently, program **execution time** is really **concerned**

Run time is denoted by $t_P(\text{instance characteristics}).$

1st Approach- Since **tp** depends upon **many factors** that are **not known** at the time a **program** is **conceived**, it is **reasonable** to **attempt** only to **estimate tp**

If the **characteristics** of the **compiler** to be used is known ,

- could have been proceeded to **determine** the **no of additions, subtractions, multiplications, divisions, compares, loads, stores** & so on, that would be made by the **code** for **P**

It can be calculated by $t_P(\text{instance characteristics}).$

$$t_P(n) = c_a ADD(n) + c_s SUB(n) + c_m MUL(n) + c_d DIV(n) + \cdots$$

- Where **n** denotes the **instance characteristics** , and
- **ca, cs, cm, cd** denote the **time** needed for an **addition**, **subtraction**, **multiplication**, **division**,

- **ADD, SUB, MUL, DIV** denotes the **functions** whose values are the **no of additions, subtractions, multiplications, divisions**
- that are used performed when the code for P is used on an instance characteristic n.

But this approach is **rarely worth** the effort since obtaining an **exact formula** is it an **impossible task**.

Since **time needed** for an **+,-,*, /** depends on the **numbers** being **added**, **subtracted**...

2nd Approach: The value of **tp(n)** can be obtained only **experimentally**

Best approach is to use the **system clock** to time the program.

This approach also had an **drawback** in a **multiuser system**

3rd Approach: counting the **number of operations** that the program performs.

- It gives **machine independent estimate**, but should have the **knowledge** of **how** to **divide the program into distinct steps**

4th Approach :**counting** only the **program steps**

A program step is a **syntactically** or **semantically meaningful program segment** whose **execution time** is **independent** of the **instance characteristics**.

The **amount of computing** represented by **one program step** may be **different** from that represented by **another step**.

- **Comment statements** count as **zero steps**,
- an **assignment statement** which does not involve any calls to other algorithms is counted as **one**.
- For ex a **simple assignment statement** of the form a=2 counts **one step**.
- And also a more **complex statement** such as a=2*b+3*c/d-e+f/g/a/b/c counts as **one step**.

The time required to execute these statements(that is counted as one step)be **independent** of the instance characteristics.

In **iterative statements** such as the ***for, while*** **and** ***repeat until*** **statements** will **consider** the **step count** only for the **control part** of the statement

The **number of steps** needed by a program to solve a particular problem instance can be determined by two methods

- **StepCount Method**
- **Tabular Column Method**

In **stepcount** method a **global variable** called ***count*** is used which has a **initial value** of **0**.

Statements to **increment *count*** by the appropriate amount are introduced into the program.

This is done so that each time a statement in the original program is executed

- ***count*** is **incremented** by the **step count** of that **statement**.

The **change** in the **value** of ***count*** by the time this program **terminates** is the number of steps executed by the algorithm

Step Count Method

```
1    Algorithm Sum(a, n)
2    {
3        s := 0.0;
4        for i := 1 to n do
5            s := s + a[i];
6        return s;
7    }
```

Algorithm 1.6 Iterative function for sum

```
1   Algorithm Sum(a, n)
2   {
3       s := 0.0;
4       count := count + 1; // count is global; it is initially zero.
5       for i := 1 to n do
6       {
7           count := count + 1; // For for
8           s := s + a[i]; count := count + 1; // For assignment
9       }
10      count := count + 1; // For last time of for
11      count := count + 1; // For the return
12      return s;
13  }
```

Algorithm 1.8 Algorithm 1.6 with count statements added

```
1   Algorithm Sum(a, n)
2   {
3       for i := 1 to n do count := count + 2;
4       count := count + 3;
5   }
```

Algorithm 1.9 Simplified version of Algorithm 1.8

$2n + 3$ steps.

```
1   Algorithm RSum(a, n)
2   {
3       count := count + 1; // For the if conditional
4       if (n ≤ 0) then
5       {
6           count := count + 1; // For the return
7           return 0.0;
8       }
9       else
10      {
11          count := count + 1;  // For the addition, function
12                               // invocation and return
13          return RSum(a, n − 1) + a[n];
14      }
15  }
```

Algorithm 1.10 Algorithm 1.7 with count statements added

When analyzing a recursive program for its step count, we often obtain a recursive formula for the step count, for example,

$$t_{\mathsf{RSum}}(n) = \begin{cases} 2 & \text{if } n = 0 \\ 2 + t_{\mathsf{RSum}}(n-1) & \text{if } n > 0 \end{cases}$$

$$\begin{aligned} t_{\mathsf{RSum}}(n) &= 2 + t_{\mathsf{RSum}}(n-1) \\ &= 2 + 2 + t_{\mathsf{RSum}}(n-2) \\ &= 2(2) + t_{\mathsf{RSum}}(n-2) \\ &\vdots \\ &= n(2) + t_{\mathsf{RSum}}(0) \\ &= 2n + 2, \qquad n \geq 0 \end{aligned}$$

So the step count for RSum (Algorithm 1.7) is $2n + 2$.

```
1   Algorithm RSum(a, n)
2   {
3       if (n ≤ 0) then return 0.0;
4       else return RSum(a, n − 1) + a[n];
5   }
```

Algorithm 1.7 Recursive function for sum

$2n + 2$.

```
1   Algorithm Add(a, b, c, m, n)
2   {
3       for i := 1 to m do
4       {
5           count := count + 1; // For 'for i'
6           for j := 1 to n do
7           {
8               count := count + 1; // For 'for j'
9               c[i, j] := a[i, j] + b[i, j];
10              count := count + 1; // For the assignment
11          }
12          count := count + 1;// For loop initialization and
13                             // last time of 'for j'
14      }
15      count := count + 1;    // For loop initialization and
16                             // last time of 'for i'
17  }
```

Algorithm 1.12 Matrix addition with counting statements

$$2mn + 2m + 1$$

```
1   Algorithm Add(a, b, c, m, n)
2   {
3       for i := 1 to m do
4           for j := 1 to n do
5               c[i, j] := a[i, j] + b[i, j];
6   }
```

gorithm 1.11 Matrix addition

```
1   Algorithm Add(a, b, c, m, n)
2   {
3       for i := 1 to m do
4       {
5           count := count + 2;
6           for j := 1 to n do
7               count := count + 2;
8       }
9       count := count + 1;
10  }
```

Algorithm 1.13 Simplified algorithm with counting only

The step count tells us how the run time for a program changes with changes in the instance characteristics.

From the step count for *Sum*, if *n* is doubled, the run time also doubles(appr)

If *n* increases by a factor of 10, the run time increases by a factor of 10 and so on.

The runtime grows linearly in n.

Sum is a linear time algorithms

- i.e the time complexity is linear in the instance characteristic *n*

One of the instance characteristics that is frequently used is the input size

The input size for the problem of summing an array with n elements is n+1, n for listing the n elements and 1 for the value of n.

Tabular Method

TABULAR METHOD

The second method to determine the step count of an algorithm is to build a table in which we list the total number of steps contributed by each statement.

First we have to identify the **no of steps per execution(s/e)** of the statement and

next the **total no of times (frequency)** each statement is executed.

The s/e of a statement is the amount by which the **count changes as a result of the execution of that statement.**

Combine these 2 quantities to get the total contribution of each statement.

Finally by adding the contributions of all statements, the step count for the entire algorithm is obtained

Statement		s/e	frequency	total steps
1	**Algorithm** Sum(a, n)	0	–	0
2	{	0	–	0
3	$s := 0.0$;	1	1	1
4	**for** $i := 1$ **to** n **do**	1	$n + 1$	$n + 1$
5	$s := s + a[i]$;	1	n	n
6	**return** s;	1	1	1
7	}	0	–	0
Total				$2n + 3$

Table 1.1 Step table for Algorithm 1.6

Statement		s/e	frequency		total steps	
			$n = 0$	$n > 0$	$n = 0$	$n > 0$
1	**Algorithm** RSum(a, n)	0	–	–	0	0
2	{					
3	**if** $(n \leq 0)$ **then**	1	1	1	1	1
4	**return** 0.0;	1	1	0	1	0
5	**else return**					
6	RSum$(a, n - 1) + a[n]$;	$1 + x$	0	1	0	$1 + x$
7	}	0	–	–	0	0
Total					2	$2 + x$

$$x = t_{\mathsf{RSum}}(n-1)$$

Table 1.2 Step table for Algorithm 1.7

Statement	s/e	frequency	total steps
1 **Algorithm** Add(a, b, c, m, n)	0	–	0
2 {	0	–	0
3 **for** $i := 1$ **to** m **do**	1	$m+1$	$m+1$
4 **for** $j := 1$ **to** n **do**	1	$m(n+1)$	$mn+m$
5 $c[i, j] := a[i, j] + b[i, j];$	1	mn	mn
6 }	0	–	0
Total			$2mn + 2m + 1$

Table 1.3 Step table for Algorithm 1.11

SUMMARY

Before identifying the step count of an algorithm, we need to know exactly which characteristics of the problem instance are to be used.

These define the variables in the expression for the step count

Ex 1)n nos to be added 2) m rows and n columns in matrix addition

A step is any computation unit that is independent of the characteristics(n,m,p,q,r)

10 additions can be one step, 100 additions can be one step, but n additions cannot.

Nor can m/2 additions, p+q subtractions, etc

3 kinds of step count

- Best-case step count-minimum no of steps that can be executed for the given parameters
- Average-case step count- average no of steps that can be executed for the given parameters
- Worst--case step count- maximum no of steps that can be executed for the given parameters

Asymptotic Notation (O)

While studying algorithms it will be interesting to characterize them according to their efficiency.

Usually interested in the order of growth of the running time of an algorithm, not in the exact running time.

This is also referred to as the asymptotic running time.

For Comparing algorithms, we need to develop a way to talk about rate of growth of functions

Asymptotic notation gives us a method for classifying functions according to their rate of growth.

Notations

Big Oh

Omega

Theta

BIG OH NOTATIONS (O)

Definition
f(n) = O(g(n)) iff there exist positive constants c and n_0 such that $f(n) \leqslant cg(n)$ for all n, $n \geqslant n_0$.

Examples

- $3n+2=O(n)$ /* $3n+2 \leqslant 4n$ for $n \geqslant 2$ */
- $3n+3=O(n)$ /* $3n+3 \leqslant 4n$ for $n \geqslant 3$ */
- $100n+6=O(n)$ /* $100n+6 \leqslant 101n$ for $n \geqslant 10$ */
- $10n^2+4n+2=O(n^2)$ /* $10n^2+4n+2 \leqslant 11n^2$ for $n \geqslant 5$ */
- $6*2^n+n^2=O(2^n)$ /* $6*2^n+n^2 \leqslant 7*2^n$ for $n \geqslant 4$ */

f(n) = O(g(n))- states only that g(n) is an upper bound on the value of f(n) for all n, n>=n0

To be more informative g(n) should be as small a function of n

O(1): constant

O(n): linear

O(n^2): quadratic

O(n^3): cubic

O(2^n): exponential

O(logn)

O(nlogn)

*FIGURE 1.7:FUNCTION VALUES (P.38)

		Instance characteristic n					
Time	Name	1	2	4	8	16	32
1	Constant	1	1	1	1	1	1
$\log n$	Logarithmic	0	1	2	3	4	5
n	Linear	1	2	4	8	16	32
$n \log n$	Log linear	0	2	8	24	64	160
n^2	Quadratic	1	4	16	64	256	1024
n^3	Cubic	1	8	64	512	4096	32768
2^n	Exponential	2	4	16	256	65536	4294967296
$n!$	Factorial	1	2	24	40326	20922789888000	26313×10^{33}

*FIGURE 1.8: PLOT OF FUNCTION VALUES (P.39)

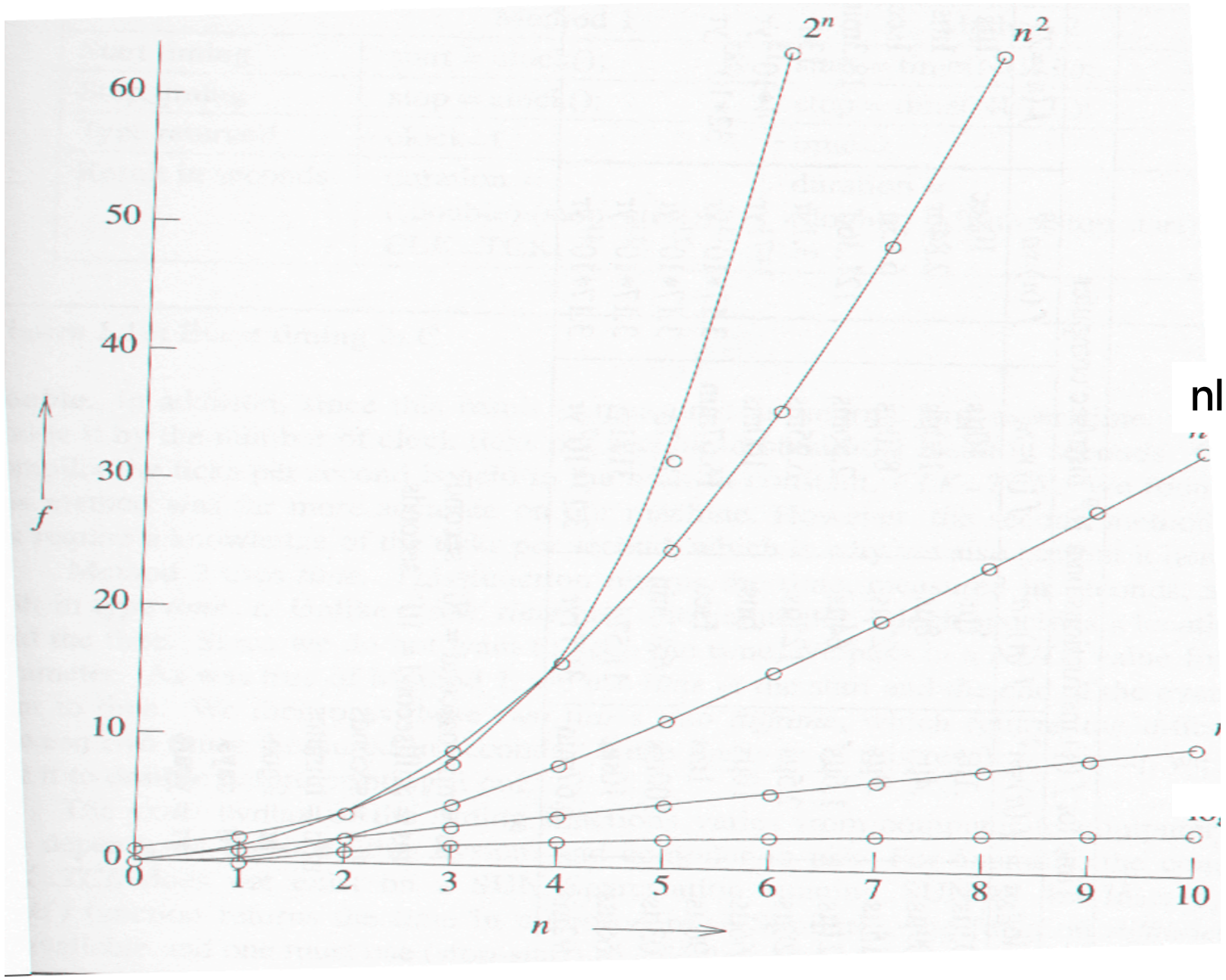


*FIGURE 1.9: TIMES ON A 1 BILLION INSTRUCTION PER SECOND COMPUTER(P.40)

	Time for $f(n)$ instructions on a 10^9 instr/sec computer						
n	$f(n)=n$	$f(n)=\log_2 n$	$f(n)=n^2$	$f(n)=n^3$	$f(n)=n^4$	$f(n)=n^{10}$	$f(n)=2^n$
10	.01μs	.03μs	.1μs	1μs	10μs	10sec	1μs
20	.02μs	.09μs	.4μs	8μs	160μs	2.84hr	1ms
30	.03μs	.15μs	.9μs	27μs	810μs	6.83d	1sec
40	.04μs	.21μs	1.6μs	64μs	2.56ms	121.36d	18.3min
50	.05μs	.28μs	2.5μs	125μs	6.25ms	3.1yr	13d
100	.10μs	.66μs	10μs	1ms	100ms	3171yr	$4*10^{13}$yr
1,000	1.00μs	9.96μs	1ms	1sec	16.67min	$3.17*10^{13}$yr	$32*10^{283}$yr
10,000	10.00μs	130.03μs	100ms	16.67min	115.7d	$3.17*10^{23}$yr	
100,000	100.00μs	1.66ms	10sec	11.57d	3171yr	$3.17*10^{33}$yr	
1,000,000	1.00ms	19.92ms	16.67min	31.71yr	$3.17*10^{7}$yr	$3.17*10^{43}$yr	

μs = microsecond = 10^{-6} seconds
ms = millisecond = 10^{-3} seconds
sec = seconds
min = minutes
hr = hours
d = days
yr = years

Definition 1.5 [Omega] The function $f(n) = \Omega(g(n))$ (read as "f of n is omega of g of n") iff there exist positive constants c and n_0 such that $f(n) \geq c * g(n)$ for all n, $n \geq n_0$. □

Examples

- 3n+2= Ω (n) /* 3n+2>=3n for n≥1 */
- 3n+3= Ω (n) /* 3n+3>=3n for n≥1 */
- 100n+6= Ω (n) /*100n+6>=100n for n≥1 */
- $10n^2+4n+2 = \Omega(n^2)$ /* $10n^2+4n+2 >= n^2$ for n≥1 */
- $6*2^n+n^2 = \Omega(2^n)$ /* $6*2^n+n^2 >= 2^n$ for n≥1*/

f(n) = omega(g(n))- states only that g(n) is an lower bound on the value of f(n) for all n, n>=n0

To be more informative g(n) should be as large a function of n

Definition 1.6 [Theta] The function $f(n) = \Theta(g(n))$ (read as "f of n is theta of g of n") iff there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all n, $n \geq n_0$. □

Examples

- 3n+2= (n) /* 3n+2>=3n for n≥2 and 3n+2<=4n for all n>=2*/
- So c1=3, c2=4 and n0=2,
- 3n+3= (n)
- $10n^2+4n+2=$ (n^2)
- $6*2^n+n^2=$ (2^n)
- 10+logn+4= (log n)

The Theta notation is more precise than both the Big oh and omega notations

The function f(n)= g(n) iff g(n) is both upper and lower bound on f(n)

Definition 1.7 [Little "oh"] The function $f(n) = o(g(n))$ (read as "f of n is little oh of g of n") iff

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Definition 1.8 [Little omega] The function $f(n) = \omega(g(n))$ (read as "f of n is little omega of g of n") iff

$$\lim_{n \to \infty} \frac{g(n)}{f(n)} = 0$$

Video Links